AI Job Retrieval & Optimization

Comparing BFS, A*, and Genetic Algorithm for AI Job Search

Ngoc Bao Tran Nguyen

Data Science & AI Project Portfolio

2026

Table of Contents

Table of Contents ***2***

List of Figures ***4***

Part 1. Formal Problem Modelling ***5***

1.1 Introduction 6

1.2 Formal Problem Definition 6

1.3 Dataset description 7

1.4 Optimization Function Design 7

1.5 Trade-off Analysis 8

Part 2. Algorithm Design and Theoretical Analysis ***10***

2.1 Uninformed search: Breadth – First Search (BFS) 10

2.2 Informed Search: A* 10

2.3 Optimization Method: Genetic Algorithm (GA) 11

Part 3. Heuristic Design ***12***

3.1 Heuristic Function Definition 12

3.2 Theoretical Justification 12

Part 4. Empirical Evaluation and Scalability ***14***

4.1 Experimental Setup 14

4.2 Comparative Results 14

4.3 Scalability Discussion 18

Part 5. Multi-Objective Optimization & Industry Insights ***20***

5.1 Identification of Three AI-Relevant Jobs 20

5.2 Detailed Job Analysis 21

5.3 Evolution of Industry Trends Cross Both Dataset 23

5.4 Genetic Algorithm Design 24

Part 6. Industry Alignment & Skill Gap Analysis ***25***

6.1 Skill Gap Analysis 25

6.2 Development Plan 25

Part 7. Critical Reflection on Algorithm Effectiveness ***26***

7.1 Strengths and Limitations of Applied Approaches 26

7.2 Optimality vs Approximation Trade-offs 26

Part 8. Conclusion ***27***

8.1 Summary of Key Findings 27

8.2 Reflection on Applied Approaches 27

Reference ***28***

List of Figures

Figure 1: Comparison among 3 searches of Sample dataset 1 15
Figure 2: GA convergence history of sample dataset 1 15
Figure 3: GA convergence curve in sample dataset 1 16
Figure 4: Comparison of 3 searches in sample dataset 2 17
Figure 5: GA convergence history in sample dataset 2 18
Figure 6: GA convergence curve in sample dataset 1 18
Figure 7: Top AI/tech job clusters in sample dataset 1 20
Figure 8: Top AI/tech job clusters in sample dataset 2 21
Figure 9: Skill frequency distribution in sample dataset 1 21
Figure 10: High-demand technologies in sample dataset 1 22
Figure 11: Skill frequency distribution in sample dataset 2 22
Figure 12: High-demand technologies in sample dataset 2 23

Executive Summary

This project investigates how different search and optimization algorithms (BFS, A*, and Genetic Algorithm) can be used to retrieve and rank AI-related job postings from large-scale datasets.

Key findings include:

- BFS and A* achieve strong solution quality on smaller datasets
- Genetic Algorithm scales efficiently to large datasets
- For large datasets, efficiency becomes more important than optimality

The analysis also reveals a clear shift in the job market from general software roles to specialized AI-focused positions.

Technologies: Python | Search Algorithms | Genetic Algorithm | Data Analysis

Part 1. Formal Problem Modelling

1.1 Introduction

In modern data-driven applications, job recommendation and retrieval systems must efficiently process large-scale datasets while balancing multiple user preferences such as skills, salary, experience level, and location. This problem naturally fits within the domain of AI search and optimisation, where the objective is to identify high-quality solutions from a vast search space under multiple constraints.

Traditional search algorithms such as Breadth-First Search (BFS) and A* provide systematic and heuristic-guided exploration of possible solutions, while Genetic Algorithms (GA) offer a population-based optimisation approach suitable for complex, multi-objective problems. These methods are particularly relevant in real-world job retrieval scenarios, where trade-offs between solution quality and computational cost must be carefully managed.

1.2 Formal Problem Definition

The AI-related job retrieval problem is formally defined as:

$$P = (S, A, s_0, G, c)$$

P – The Problem

The problem is to retrieve and rank AI-related job postings from large-scale job advertisement datasets, and also to rank them according to how well they match candidate preferences such as skills, salary, experience, and location.

S – State Space

The state space consists of all possible subsets of job postings generated during the search process. Each state represents a candidate set of jobs after applying one or more filtering or ranking operations.

A - Actions

Actions represent the operations that transform one state into another by applying filtering and ranking steps to the dataset. These include filtering job postings based on AI-related keywords (e.g., "AI"), removing records with missing salary information, filtering by experience level, work type, and location, and ranking results according to relevance. Each action reduces or refines the dataset, guiding the search toward goal states.

s_0 – Initial Sate

The initial state is the complete dataset containing all job advertisements before any filtering or ranking has been applied.

G – Goal Set

States that satisfy predefined criteria, including:

- Containing AI-related jobs
- Having a manageable result size (e.g., ≤ 200 jobs)

- Maintaining sufficient data quality (e.g., salary and experience available)

c – Cost Function

A function that evaluates the quality of a state based on how well it matches user preferences. Lower cost (or higher score) indicates better alignment with desired job characteristics.

1.3 Dataset description

Two datasets were used to evaluate the search and optimisation approaches:

Dataset 1:

- Size: ~19,000 job postings
- Source: Structured job dataset with limited AI-specific labeling
- Key features: title, location, salary, eligibility (experience level), term (work type), IT flag

Dataset 2:

- Size: 123,849 job postings
- Source: Recent large-scale job dataset (2023–2024)
- Key features: job ID, title, description, salary fields (min, max, normalized), experience level, work type, location, additional metadata (views, applications, etc.)

Data preprocessing

To ensure consistency across datasets:

- Columns were standardised (e.g., salary fields unified into salary_final)
- Missing or noisy values were cleaned
- Text fields were normalised for filtering (e.g., AI keyword detection)

Potential Biases

- Dataset 1 under-represents AI jobs (older data)
- Salary information is incomplete or inconsistent
- Filtering rules (e.g., keyword-based AI detection) may introduce bias
- Large dataset (Dataset 2) may favour scalability over precision

1.4 Optimization Function Design

The objective is to maximise the quality of retrieved job sets based on multiple criteria:

$$\max f(j) = w_1 SalaryScore + w_2 SalaryAvailable + w_3 FullTimeScore + w_4 ExperienceScore + w_5 LocationScore + w_6 FocusScore + w_7 ActionDiversity$$

Each component captures a different aspect of job quality:

- SalaryScore: Measures the average salary of the job set, encouraging higher-paying opportunities.
- SalaryAvailable: Rewards datasets with fewer missing salary values, improving data reliability.
- FullTimeScore: Prioritises full-time roles, which are generally more desirable for stable employment.
- ExperienceScore: Measures the availability and relevance of experience level information.
- LocationScore: Rewards job postings with valid and informative location data.
- FocusScore**:** Encourages smaller, more targeted result sets, avoiding overly broad or noisy outputs.
- ActionDiversity: Rewards solutions that apply a meaningful combination of different filtering actions, promoting exploration of the search space.

The weights w_1 to w_7 control the relative importance of each factor. This formulation enables the system to balance multiple objectives simultaneously, rather than optimising a single criterion.

1.5 Trade-off Analysis

Weight Selection

The selection of weights plays a critical role in shaping optimisation outcomes:

- Increasing SalaryScore weight prioritises high-paying jobs but may reduce dataset size or exclude relevant roles.
- Increasing SalaryAvailable weight improves data completeness but may discard useful jobs with missing salary fields.
- Emphasising FullTimeScore and ExperienceScore ensures higher-quality job information but reduces flexibility.
- A higher FocusScore weight leads to smaller, more precise result sets but risks over-filtering.
- Increasing ActionDiversity promotes broader exploration but may introduce less consistent solutions.

Therefore, weight tuning is necessary to achieve a balance between relevance, quality, and coverage.

Optimisation Quality vs Computational Cost (A* vs GA)

The choice of optimisation algorithm introduces important trade-offs:

A*

- Produces more structured and interpretable solutions
- Efficient when guided by a well-designed heuristic
- However, it becomes computationally expensive as dataset size grows

Genetic Algorithm (GA)

- Efficient for large-scale datasets (e.g., Dataset 2)
- Can optimise complex multi-objective functions like the one defined above
- However, it does not guarantee optimality and may converge to local optima

Part 2. Algorithm Design and Theoretical Analysis

To solve the AI job retrieval problem, three search strategies are selected: Breadth-First Search (BFS) as the uninformed search method, A* as the informed search method, and Genetic Algorithm (GA) as the optimization method. These algorithms are compared in terms of time complexity, space complexity, completeness, optimality, and suitability for large-scale job advertisement datasets.

2.1 Uninformed search: Breadth – First Search (BFS)

BFS explores the search space level by level, expanding all states at the current depth before moving to the next level. In this problem, each state represents a subset of job postings after applying filtering actions (e.g., AI-related filtering, salary filtering).

- **Time complexity:** $O(b^d)$ where b is the branching factor and d is the depth of the shallowest goal.
- **Space complexity:** $O(b^d)$, because BFS stores all generated nodes at each level.
- **Completeness:** BFS is complete if the branching factor is finite.
- **Optimality:** when all actions have equal cost, as it finds the shallowest goal state.
- **Suitability for large datasets:** Limited. BFS is easy to implement and useful as a baseline, but it becomes expensive in memory and time when the dataset is large because the number of possible filtered states grows quickly.

2.2 Informed Search: A*

A* improves search efficiency by using both the path cost *g(n)* and a heuristic estimate *h(n)*. For job retrieval, *g(n)* may represent the accumulated mismatch cost after applying filters, while *h(n)* estimates how close the current state is to the ideal AI Engineer job set.

The heuristic function is formulated as:

$$f(n) = g(n) + h(n)$$

$g(n)$: represents the accumulated score of the current state after applying a sequence of filtering actions.

$h(n)$: Estimates the potential improvement that can still be achieved from the current state toward a goal state.

Cost Accumulation and Heuristic Design

In this problem:

$g(n)$: reflects the current quality of the job subset (e.g., salary, completeness, relevance).

$h(n)$: estimates how much the result can be further improved (e.g., reducing size, improving salary quality, increasing relevance).

The heuristic helps prioritise more promising states, reducing unnecessary exploration compared to BFS.

Theoretical Properties

- Completeness: A* is complete if the heuristic is admissible.
- Optimality: A* is optimal if the heuristic does not overestimate the remaining cost.

- Time Complexity: Exponential in the worst case, but typically much lower than BFS with a good heuristic.
- Space Complexity: High, as it stores all generated nodes in memory.

2.3 Optimization Method: Genetic Algorithm (GA)

Genetic Algorithm is used to optimize job ranking under multiple objectives. Instead of exploring states systematically like BFS or A*, GA evolves a population of candidate solutions over several generations.

Chromosome Representation

Each chromosome represents a candidate solution as a sequence of filtering actions. In this problem, a chromosome encodes which actions are applied (e.g., AI filter, salary filter, experience filter), effectively defining a transformation from the initial dataset to a refined job subset.

Fitness Function

The fitness of each chromosome is evaluated using the multi-objective optimisation function:

$$f(j) = w_1 SalaryScore + w_2 SalaryAvailable + w_3 FullTimeScore + w_4 ExperienceScore + w_5 LocationScore + w_6 FocusScore + w_7 ActionDiversity$$

Selection, Crossover, Mutation

- Selection: Higher-fitness individuals are more likely to be selected for reproduction.
- Crossover: Combines parts of two parent solutions to create new offspring, enabling exploration of new action combinations.
- Mutation: Randomly alters parts of a chromosome to maintain diversity and avoid premature convergence.

Termination Criteria

The algorithm stops when a maximum number of generations is reached, or the fitness score converges (no significant improvement over time).

Comparison and Justification

BFS is useful as a simple uninformed baseline because it does not require domain knowledge, but it scales poorly. A* is more effective because it uses heuristic guidance to prioritize promising job subsets, making it more suitable for targeted job retrieval. Genetic Algorithm is particularly appropriate for this assignment because the problem is naturally multi-objective, involving trade-offs between skill match, salary, experience, and location. Among the three methods, GA is likely to be the most scalable for large job datasets, while A* provides the strongest search guidance when a good heuristic is available.

Part 3. Heuristic Design

3.1 Heuristic Function Definition

For A*, the heuristic is defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ represents the current quality score of the job subset after applying a sequence of actions
- $h(n)$ estimates the potential improvement that can still be achieved from the current state

In this problem, the heuristic function is designed to reflect how much the current solution can be improved toward an ideal job set. Specifically, the heuristic considers:

- Remaining potential to increase salary quality
- Improvement in data completeness (e.g., missing salary or experience fields)
- Reduction of result size toward a desired range (focused results)
- Additional filtering actions that may improve relevance (e.g., AI-related filtering)

Thus, $h(n)$ provides an estimate of the "gap" between the current state and a high-quality goal state.

3.2 Theoretical Justification

Admissibility:

A heuristic is admissible if it never overestimates the true cost to reach a goal. In this problem, strict admissibility is not guaranteed because:

- The objective is to maximise a score, not minimise cost
- The heuristic estimates potential improvement rather than exact remaining cost
- Some improvements (e.g., salary increase after filtering) are uncertain and data-dependent

Therefore, the heuristic may overestimate achievable improvements, meaning A* does not strictly guarantee optimality.

Consistency:

A heuristic is consistent if the estimated cost satisfies the triangle inequality:

$$h(n) \leq c(n, n') + h(n')$$

In this case, consistency is also not strictly guaranteed because:

- Applying a filtering action can cause non-linear changes in the dataset (e.g., removing many records at once)
- The effect of actions is not uniform or predictable
- Heuristic values may change irregularly between states

However, the heuristic is designed to be reasonably monotonic in practice, guiding the search toward better states even without strict theoretical guarantees.

Suitability for Large Datasets:

Despite lacking strict admissibility and consistency, the heuristic is well-suited for large-scale problems:

- It prioritises promising states, reducing unnecessary exploration compared to BFS
- It aligns directly with the multi-objective scoring function, ensuring consistency between search and evaluation
- It enables A* to handle moderately large datasets more efficiently

However, for very large datasets, the search space remains too large for A* to scale effectively. In such cases, heuristic-guided search still incurs high computational cost, making GA a more practical alternative.

Part 4. Empirical Evaluation and Scalability

4.1 Experimental Setup

Datasets Used

Two datasets are used to evaluate the performance and scalability of the search algorithms:

- **Sample Dataset 1**:

A smaller dataset containing 19,001 job postings. This dataset includes structured fields such as title, location, salary, experience level, and work type. However, as this dataset is relatively older, AI-related roles are not explicitly indicated in the job titles. Therefore, an it_flag feature was used as a proxy to identify AI-related or technical jobs during filtering. Due to its manageable size, all algorithms (BFS, A*, and GA) are applied directly to the full dataset.

- **Sample Dataset 2**:

A large-scale dataset containing 123,849 job postings with richer attributes, including detailed salary fields, experience level, work type, and additional metadata. To analyse scalability, experiments are conducted on both sampled subsets of the dataset and the full dataset.

Before applying the search algorithms, both datasets are preprocessed to ensure consistency:

- Salary fields were cleaned and unified into a single salary_final feature
- Text fields were standardised to support filtering (e.g., AI-related keyword detection)
- Missing or noisy values were handled to improve data quality

4.2 Comparative Results

- **Sample dataset 1**

The performance of BFS, A*, and GA on Dataset 1 (19,001 job postings) is summarised in Figure 1. The comparison focuses on runtime, memory usage, node expansions, and solution quality.

Performance Comparison

Both BFS and A* produced identical results in terms of solution quality. They achieved the highest-ranking quality score of 89.03, with an average salary of 400,000 and a final result size of 6 jobs. This indicates that both algorithms are able to identify highly optimal and focused job subsets.

However, their computational behaviour differs slightly. BFS had a runtime of approximately 44.6 seconds, while A* took slightly longer at 47.6 seconds, likely due to the overhead of heuristic evaluation.

Both algorithms expanded the same number of nodes (2094) and found the same number of goal states (415), suggesting that the heuristic used in A* did not significantly reduce the search space for this dataset.

In contrast, the Genetic Algorithm (GA) demonstrated different behaviour. It achieves a lower ranking quality score (81.08) and a lower average salary (271,111). The result size is slightly larger (9 jobs), indicating less focused filtering. However, GA required fewer node expansions (900) and had the fastest runtime (~41.9 seconds), demonstrating higher computational efficiency.

```
===== FINAL COMPARISON: BFS vs A* vs GA =====
```

	algorithm	dataset_name	dataset_size	ai_jobs_found	missing_salary_final	runtime_seconds	memory_mb	node_expansions	num_goal_states_found	best_jobs_remaining	best_avg_salary	best_ranking_quality	best_depth	best_actions
0	BFS	Dataset 1	19001	3786	18075	44.5946	12.0984	2094	415	6	400000.00	89.03	6	['filter_location_available', 'filter_ai_relat...
1	A*	Dataset 1	19001	3786	18075	47.6145	11.2909	2094	415	6	400000.00	89.03	6	['filter_location_available', 'filter_ai_relat...
2	GA	Dataset 1	19001	3786	18075	41.9308	30.6459	900	658	9	271111.11	81.08	4	['filter_work_type_full_time', 'filter_experie...

Figure 1: Comparison among 3 searches of Sample dataset 1

Trade-off Between Solution Quality and Practical Usability

The results highlight an important trade-off between optimality and practical usability.

While BFS and A* achieve slightly higher ranking scores, they produce extremely small result sets (6 jobs). Such highly filtered outputs, although optimal in score, limit their usefulness for downstream analysis tasks. With very few data points, it becomes difficult to perform meaningful analysis such as skill distribution, clustering, or identifying industry patterns.

In contrast, GA provides a more balanced outcome. Although its ranking score is slightly lower, it maintains a larger and more diverse result set (9 jobs), which is more suitable for further analysis. This balance between solution quality and dataset usability makes GA more appropriate for multi-objective analysis and insight generation in Part 5.

GA Convergence Analysis

The best fitness increases rapidly from an initial value of approximately 231.52 to 282.25 within the first few generations (around generation 4). After this point, the best fitness remains relatively stable, indicating that the algorithm quickly identifies a strong candidate solution but is unable to significantly improve beyond it.

```
GA Convergence History:
```

	generation	best_fitness	avg_fitness	best_ranking_quality	best_jobs_remaining	best_actions
0	1	231.5169	-866649.171657	65.4944	2	['filter_salary_above_median', 'filter_ai_rela...
1	2	278.9134	-433245.322470	79.7996	80	['filter_work_type_full_time', 'filter_salary_...
2	3	278.9134	-133138.043290	79.7996	80	['filter_work_type_full_time', 'filter_salary_...

```
Best GA Actions: ['filter_work_type_full_time', 'filter_experience_available', 'filter_ai_related', 'filter_salary_above_median']
Best GA Ranking Quality: 81.0813
Best GA Fitness: 273.1204
Best GA Avg Salary: 271111.11
```

Figure 2: GA convergence history of sample dataset 1

Meanwhile, the average fitness fluctuates considerably across generations and remains negative throughout most of the process. This suggests that a large portion of the population consists of low-quality solutions, and overall population improvement is limited.

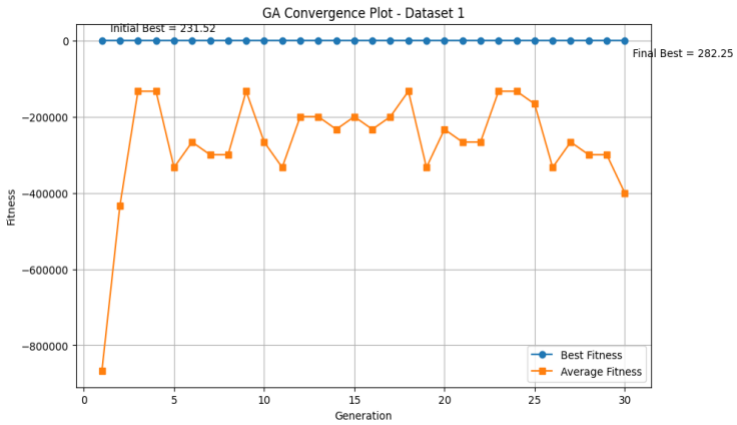

Figure 3: GA convergence curve in sample dataset 1

This pattern indicates premature convergence, where the GA converges early to a local optimum and loses diversity. Possible reasons include:

- Limited diversity in the initial population
- Insufficient mutation rate to explore new solutions
- A relatively constrained search space in sample dataset 1

Despite this limitation, GA is still able to efficiently identify a reasonably good solution with lower computational cost compared to BFS and A*.

Overall, BFS and A* achieve the highest solution quality but are less practical due to over-filtering and higher computational cost. GA, while slightly less optimal, provides a better balance between efficiency and usability, making it the preferred approach for subsequent analysis tasks.

- **Sample dataset 2**

The performance of BFS, A*, and Genetic Algorithm (GA) was evaluated on Dataset 2 across increasing dataset sizes, ranging from 10,000 to the full dataset of 123,849 job postings.

Performance Comparison

The performance of BFS, A*, and GA differs significantly as dataset size increases.

In terms of runtime, BFS and A* show substantial growth as the dataset scales. BFS increases from 120.10 seconds (10k) to 434.50 seconds (full dataset), while A* increases from 148.41 seconds to 522.15 seconds. In contrast, GA remains significantly faster, with runtime increasing only from 46.76 seconds to 79.14 seconds.

In terms of memory usage, BFS and A* consume considerably more resources as dataset size grows, reaching over 300 MB and 245 MB respectively. GA maintains much lower memory usage, remaining only 25 MB even for the full dataset.

Regarding node expansions, BFS and A* expand an increasing number of nodes (from 2810 to 4415), reflecting higher computational effort. GA does not rely on node expansion in the same way, instead using a fixed number of fitness evaluations (1500), making its computational cost more stable.

Notably, BFS and A* exhibit very similar behaviour across all evaluation metrics. Both algorithms achieve nearly identical ranking scores (approaching 100 for larger datasets) and show comparable trends in node

expansions. This suggests that the heuristic used in A* does not significantly reduce the search space compared to BFS in this problem setting. As a result, A* behaves similarly to uninformed search, particularly when many states yield comparable solution quality.

Overall, GA demonstrates significantly better performance efficiency compared to BFS and A*, particularly in large-scale scenarios.

===== COMPACT COMPARISON (4 METRICS) =====

	Dataset Size	BFS Runtime (s)	A* Runtime (s)	GA Runtime (s)	BFS Memory (MB)	A* Memory (MB)	GA Memory (MB)	BFS Node Exp.	A* Node Exp.	GA Fitness Eval.	BFS Ranking	A* Ranking	GA Ranking
0	10000	120.095	148.413	46.756	28.139	28.378	11.013	2810	2810	1500	96.0	96.0	94.0
1	30000	190.404	239.640	57.427	75.492	71.625	13.876	3629	3629	1500	100.0	100.0	100.0
2	50000	256.627	311.291	60.014	124.758	111.498	16.080	3969	3969	1500	100.0	100.0	100.0
3	100000	385.438	464.176	76.816	246.273	210.499	22.677	4343	4343	1500	100.0	100.0	100.0
4	123849	434.503	522.154	79.138	301.307	245.166	25.330	4415	4415	1500	100.0	100.0	100.0

Figure 4: Comparison of 3 searches in sample dataset 2

Solution Quality Comparison

Despite large differences in computational cost, all three algorithms achieve very similar solution quality:

- BFS and A* consistently achieve the highest-ranking score (96–100)
- GA achieves slightly lower but still comparable scores (94–100)

As dataset size increases beyond 30,000 records, all methods converge to a maximum ranking score of 100, indicating that high-quality solutions can be found regardless of the search strategy.

This suggests that, for large datasets, the advantage of BFS and A* in achieving slightly higher optimality becomes negligible.

Trade-off Between Optimality and Scalability

BFS and A* provide guaranteed or near-optimal solutions, but at the cost of rapidly increasing computational requirements. As dataset size grows, these methods become less practical due to high runtime and memory usage.

In contrast, GA sacrifices a small degree of optimality but offers significantly better efficiency and scalability. This makes GA more suitable for real-world applications involving large datasets.

GA Convergence Analysis

The convergence behaviour of GA is shown in Figure X.

For smaller datasets (e.g., 10,000 records), GA shows a modest improvement in fitness, increasing from 317.18 to 322.33. However, for larger datasets (30,000 and above), the fitness remains constant at 345.00, with no observable improvement across generations.

Additionally, convergence occurs very early (e.g., generation 2–23), indicating that GA quickly identifies a stable solution but does not further refine it.

GA Convergence History:

	generation	best_fitness	avg_fitness	best_ranking_quality	best_jobs_remaining	best_actions
0	1	345.0	270.729293	100.0	7	['filter_ai_related', 'filter_experience_avail...
1	2	345.0	310.481385	100.0	7	['filter_ai_related', 'filter_experience_avail...
2	3	345.0	319.035642	100.0	7	['filter_ai_related', 'filter_experience_avail...

Figure 5: GA convergence history in sample dataset 2

This pattern suggests early convergence and limited exploration, likely due to:

- A relatively saturated fitness landscape, where many solutions achieve similar scores
- Limited diversity in the population
- Fixed number of fitness evaluations

Despite this, the consistently high fitness values demonstrate that GA is still able to efficiently identify strong solutions without extensive search.

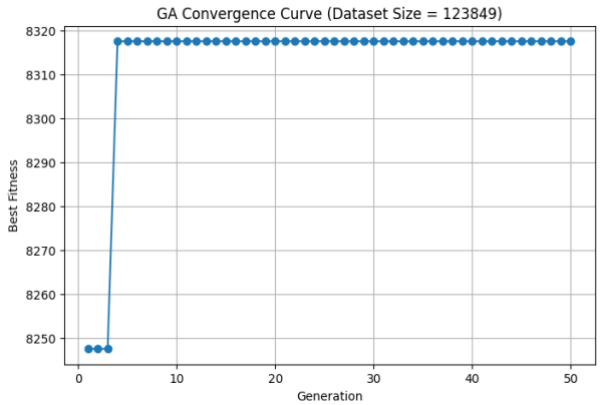

Figure 6: GA convergence curve in sample dataset 1

Overall, Dataset 2 highlights the limitations of exhaustive and heuristic search methods in large-scale scenarios. While BFS and A* maintain high solution quality, their computational cost increases significantly with dataset size. In contrast, GA provides a scalable and efficient alternative, achieving comparable performance with substantially lower resource requirements.

4.3 Scalability Discussion

Comparison Between Dataset 1 and Dataset 2

For Dataset 1 (19,001 records), all three algorithms are computationally feasible. BFS and A* are able to fully explore the search space within reasonable runtime and memory limits, achieving the highest-ranking quality. GA also performs efficiently, although with slightly lower solution quality.

However, when scaling to Dataset 2 (up to 123,849 records), clear differences emerge. BFS and A* experience substantial increases in runtime and memory usage, with runtime exceeding 400–500 seconds and memory usage reaching over 300 MB. This reflects the rapid growth of the search space, making exhaustive and heuristic search increasingly expensive.

In contrast, GA maintains stable performance across both datasets. Its runtime increases only moderately, and memory usage remains low. This is because GA evaluates a fixed number of candidate solutions rather than exploring the entire search space.

Algorithm Behaviour Across Scales

The comparison reveals distinct behaviours for each algorithm:

- **BFS and A***

- Perform well on smaller datasets, achieving optimal or near-optimal solutions
- Show significant degradation in efficiency as dataset size increases
- Exhibit similar behaviour across both datasets, indicating limited benefit from heuristic guidance

- **Genetic Algorithm (GA)**

- Provides slightly lower but still high-quality solutions
- Maintains stable runtime and memory usage across both datasets
- Scales effectively to large datasets without exponential growth in computational cost

Key Insights

Several important insights can be drawn from the cross-dataset comparison:

- Scalability limitation of BFS and A*: Although BFS and A* perform well on small datasets, their computational cost grows rapidly with dataset size, making them unsuitable for large-scale applications.
- Limited impact of heuristic design: The similar behaviour of BFS and A* across both datasets suggests that the heuristic used in A* does not significantly improve search efficiency, particularly when many states yield similar solution quality.
- Robust scalability of GA: GA demonstrates consistent performance across both datasets, highlighting its suitability for large-scale, multi-objective optimisation problems.
- Diminishing returns in solution quality: While BFS and A* achieve slightly higher-ranking scores in Dataset 1, this advantage becomes negligible in Dataset 2, where all algorithms reach similar performance levels.

Part 5. Multi-Objective Optimization & Industry Insights

5.1 Identification of Three AI-Relevant Jobs

- **Sample Dataset 1**

Based on the optimised results obtained using GA, three representative AI/tech-relevant job categories are identified from the final filtered dataset. These roles are selected based on frequency, relevance to technical skills, and alignment with industry demand**:** Software Developer, Web Developer, and C/C++ Software Engineer

These roles are among the most frequently occurring job clusters and reflect core areas of demand in the technology job market.

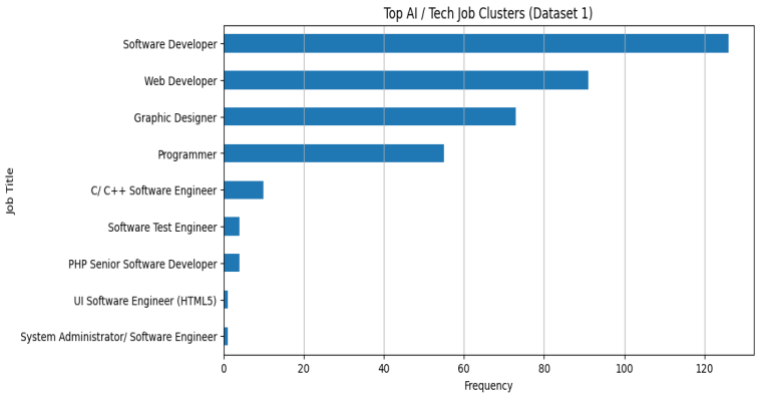

Figure 7: Top AI/tech job clusters in sample dataset 1

- **Sample Dataset 2**

Based on the optimised results obtained using the GA on the full sample dataset 2 (123,849 records), three representative AI - related roles are identified: Principal AI/ML Data Scientist, Senior Distributed Systems Engineer (AI Infrastructure), and Principal Applied AI Engineer

These roles are selected as they reflect key areas of the modern AI ecosystem, including data science, AI system infrastructure, and applied AI engineering.

Top job clusters:

title	count
Sr. Manager Solutions Architecture - Generative AI, AWS Startups	1
Principal AI/ML Data Scientist	1
Modeling, Simulation & Training AI Solution Architect	1
Senior Software Engineer, AI Developer Tools - Cloud Software	1
Senior Distributed Systems Engineer, AI Infrastructure	1
Senior Product Manager - DDoS & AI	1
Corporate Counsel - Global Generative AI Company	1
Data Scientist - AI Investment	1
Principal Product Manager, Confluence AI	1
Principal Applied AI Engineer	1
AI/ML Engineering Manager	1
Senior Staff Engineer - Generative AI(Handson Coding, L7)	1
Sr. Systems Development Eng (AWS Generative AI & ML Servers)	1
Gen-AI Full Stack Lead Engineer	1
Senior Applied Scientist, Conversational AI ModEling and Learning	1
Senior Solutions Architect - Analytics & AI, AGS-NAMER-US-COV-SPEC-SA-Data&AI-Analytics	1
Sr Data Engineer , Customer Engagement Technology Generative AI	1
Principal Product Manager, Gen AI	1
Artificial Intelligence Engineer ($225,000 - $300,000)	1
Artificial Intelligence Engineer	1
AI Solutions Architect	1

Figure 8: Top AI/tech job clusters in sample dataset 2

5.2 Detailed Job Analysis

- **Sample Dataset 1**

Software Developer

Software Developers are responsible for designing, developing, and maintaining software systems. They often contribute to building applications that integrate machine learning models, backend systems, and data processing pipelines.

Strong emphasis on programming, database management, and problem solving, as reflected in the high frequency of these skills. Knowledge of languages such as C++, Java, and SQL are commonly required.

Typically requires a degree in Computer Science or a related field, along with experience in software development and system design.

This role shows the highest frequency (126 jobs), indicating strong and consistent demand. The prominence of programming and database skills suggests that foundational software engineering remains critical in AI/tech-related roles.

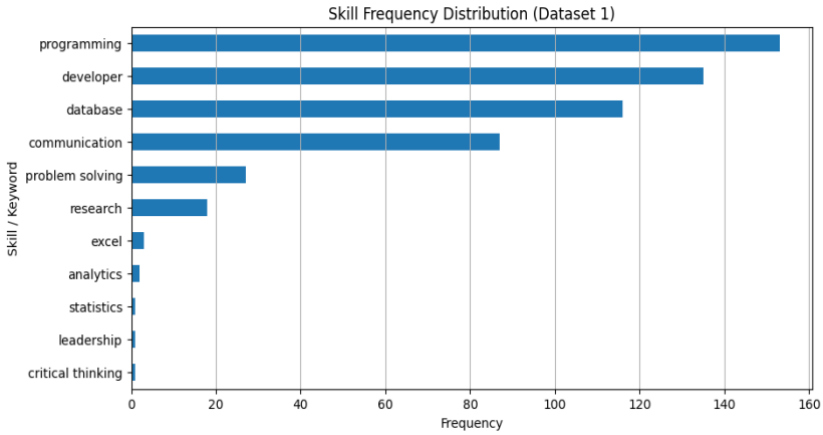

Figure 9: Skill frequency distribution in sample dataset 1

Web Developer

Web Developers focus on building and maintaining web-based applications. In AI/tech-driven systems, they often develop user interfaces and integrate backend services into web platforms.

Key skills include programming, communication, and database management. Familiarity with web technologies and backend integration is essential. Requires technical training in web development, often supported by experience with frontend and backend frameworks.

With 91 job postings, this role reflects strong demand for web-based systems that support data-driven and AI/tech-enabled applications.

C/C++ Software Engineer

C/C++ Software Engineers develop high-performance systems, often used in low-level programming, embedded systems, and performance-critical applications such as AI infrastructure and systems programming.

Strong emphasis on C++ programming, system-level thinking, and problem solving. Knowledge of Linux environments is also important.

Typically requires advanced programming expertise and strong understanding of system architecture.

Although smaller in number (10 jobs), this role highlights demand for specialised, high-performance computing skills, particularly relevant for AI systems and backend infrastructure.

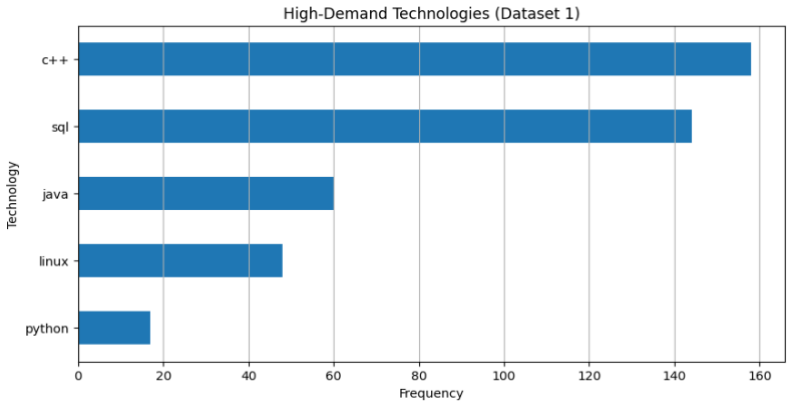

Figure 10: High-demand technologies in sample dataset 1

- **Sample Dataset 2**

Principal AI/ML Data Scientist

This role focuses on developing machine learning models, analysing large-scale datasets, and generating insights to support decision-making. It often involves building predictive models and working closely with business stakeholders.

Key skills include machine learning, data science, analytics, and statistics, along with strong programming capabilities. Typically requires advanced education, or researching in data science, computer science, or related fields, along with experience in ML model development.

The presence of this role highlights strong demand for data-driven AI solutions and advanced analytical capabilities.

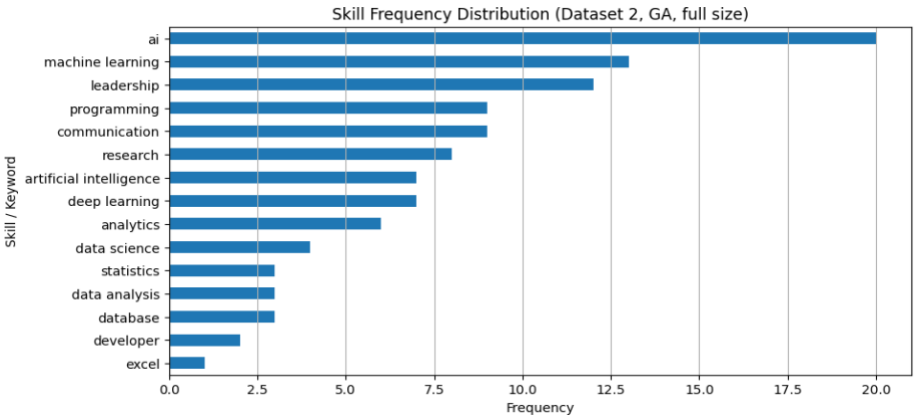

Figure 11: Skill frequency distribution in sample dataset 2

Senior Distributed Systems Engineer (AI Infrastructure)

This role focuses on designing and maintaining large-scale infrastructure systems that support AI applications, including distributed computing and cloud-based platforms.

Strong emphasis on AI, distributed systems, programming, and system architecture. Technologies such as AWS, Spark, and Hadoop are commonly required. Requires significant experience in system engineering, cloud platforms, and backend infrastructure development.

This role reflects the growing importance of scalable infrastructure to support AI workloads, particularly in large organisations.

Principal Applied AI Engineer

Applied AI Engineers focus on integrating AI models into real-world applications, including deploying models, optimising performance, and building AI-driven products.

Key skills include deep learning, AI engineering, programming, and experience with frameworks such as TensorFlow and PyTorch. Requires strong technical expertise in both software engineering and machine learning, often with industry experience.

This role highlights the shift from research-focused AI toward production-level AI systems and applications.

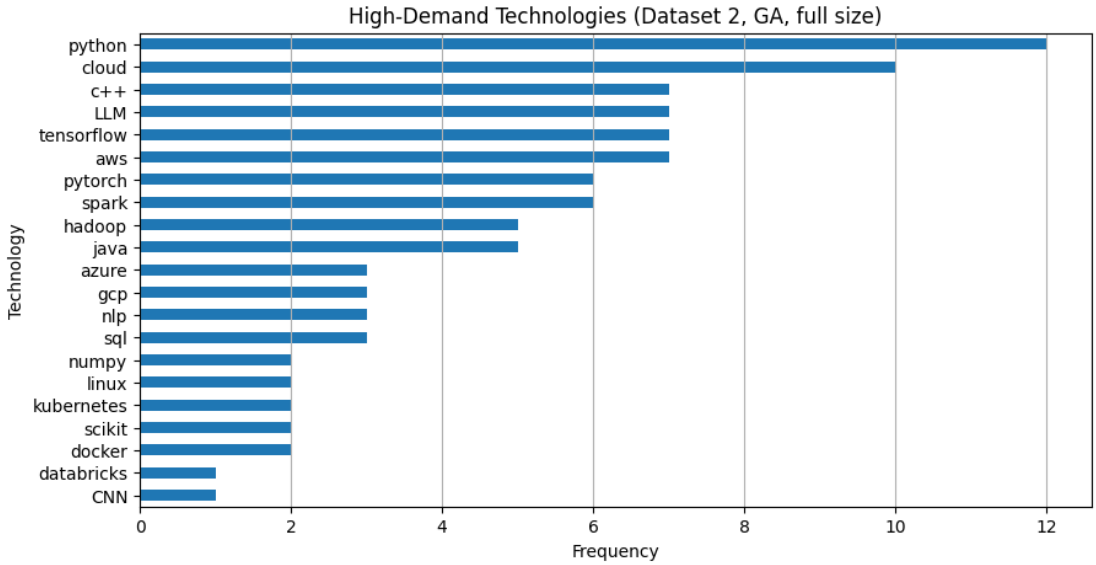

Figure 12: High-demand technologies in sample dataset 2

5.3 Evolution of Industry Trends Cross Both Dataset

A comparison between Dataset 1 and Dataset 2 reveals a clear evolution in the AI and technology job market over time.

In Dataset 1, which represents an older job market, roles are primarily centred around general software engineering positions such as Software Developer and Web Developer. The required skills are dominated by fundamental programming and database knowledge, with technologies such as C++, SQL, and Java being most prominent. AI-specific skills and tools are relatively limited, and AI-related roles are not explicitly defined, requiring the use of proxy indicators such as the it_flag.

In contrast, Dataset 2 reflects a modern and highly specialised AI job market. Job roles are explicitly AI-focused, including positions such as AI/ML Data Scientist, Applied AI Engineer, and AI Infrastructure Engineer. The required skills shift toward machine learning, deep learning, and AI engineering, along with the integration of cloud platforms and large-scale data systems. Technologies such as Python, TensorFlow, PyTorch, and Large Language Models (LLMs) become dominant, indicating the increasing importance of AI frameworks and scalable computing environments.

Furthermore, the industry shows a transition from generalist roles to specialised, high-impact positions, with greater emphasis on experience, leadership, and real-world deployment of AI systems. Salary structures also become more standardised, reflecting a more mature and competitive market.

Overall, this progression demonstrates how the technology industry has evolved from traditional software development toward a data-driven, AI-centric ecosystem, where advanced technical skills, scalable infrastructure, and applied AI capabilities are essential.

5.4 Genetic Algorithm Design

The GA is used as the optimisation method for both datasets because it is well suited to multi-objective job retrieval. In both cases, each chromosome represents a candidate filtering strategy, but the encoding differs slightly between datasets. For Dataset 1, a chromosome is an ordered sequence of filtering actions, while for Dataset 2, it is encoded as a binary vector indicating whether each action is selected. In both implementations, the fitness function is aligned with the ranking framework and evaluates salary quality, salary availability, experience information, location completeness, full-time ratio, result focus, and action diversity. Strong penalties are applied to invalid solutions, especially when no jobs are returned or when AI-related filtering is not included.

Across both datasets, GA evolves solutions through selection, crossover, and mutation to identify high-quality job subsets efficiently. Elite solutions are preserved in each generation, while new offspring are generated by recombining and mutating parent chromosomes. For Dataset 1, mutation modifies the action sequence through add, remove, replace, or shuffle operations, whereas for Dataset 2 it flips binary genes while always keeping the AI-related filter active. Overall, GA converges quickly in both datasets, showing that it can efficiently identify strong candidate solutions. However, the convergence behaviour also suggests limited diversity after early generations, which explains why improvement becomes small once a good solution has been found.

Part 6. Industry Alignment & Skill Gap Analysis

6.1 Skill Gap Analysis

Based on the analysis of both datasets, current industry demand is strongly concentrated around core technical skills such as programming, database management, and software development, with increasing emphasis on AI-related capabilities such as machine learning, deep learning, and natural language processing. In Dataset 2, high-value roles further require a combination of technical expertise and leadership or strategic thinking, indicating a shift towards hybrid skill profiles.

Comparing these trends with a typical data science or AI learner profile, there is often a gap in three key areas. First, while foundational programming skills may be present, advanced AI-specific competencies (e.g., deep learning, NLP, large-scale systems) are less consistently developed. Second, there is a noticeable gap in industry-oriented skills such as system design, real-world problem solving, and deployment in cloud environments. Third, soft skills such as leadership, communication, and research capability are increasingly important in high-ranking roles but are often underdeveloped.

This gap suggests that being technically competent alone is insufficient for achieving high-quality job outcomes. Instead, industry demand favours candidates who can integrate technical knowledge with practical application and strategic thinking.

6.2 Development Plan

To address these gaps, a structured and measurable development plan is required. In the short term, focus should be placed on strengthening core AI skills, including machine learning frameworks (e.g., TensorFlow/PyTorch), NLP techniques, and data processing pipelines. This can be measured through completion of targeted projects such as building end-to-end AI systems or deploying models in cloud environments.

In the medium term, the focus should shift towards system-level understanding and real-world application. This includes learning about scalable architectures, retrieval-augmented generation (RAG), and model deployment, which are directly aligned with current industry trends. Completing at least one capstone-level project that integrates multiple components (data, model, deployment) would provide measurable progress.

In the long term, development should extend beyond technical skills to include leadership, communication, and domain knowledge. This can be achieved through collaborative projects, presentations, or internships, where the ability to translate technical solutions into business value is essential.

Part 7. Critical Reflection on Algorithm Effectiveness

7.1 Strengths and Limitations of Applied Approaches

In this study, three algorithms - BFS, A*, and GA - demonstrate distinct strengths and limitations when applied to the job retrieval problem.

BFS provides a systematic and complete exploration of the search space, ensuring that all possible combinations of filtering actions are considered. This guarantees that high-quality solutions can be found. However, BFS suffers from high computational cost and memory usage, particularly as dataset size increases. This makes it impractical for large-scale applications.

A* improves upon BFS by incorporating heuristic guidance to prioritise promising states. In theory, this allows for more efficient search. However, in this study, A* exhibited behaviour like BFS across both datasets. This indicates that the heuristic function had limited discriminative power, reducing its effectiveness in guiding the search. As a result, A* did not significantly improve efficiency compared to BFS.

In contrast, the GA demonstrated strong performance in terms of scalability and efficiency. GA effectively handles the multi-objective nature of the problem by optimising multiple criteria simultaneously. It maintains low runtime and memory usage even for large datasets. However, GA does not guarantee optimal solutions and may converge prematurely, as observed in the convergence analysis. This highlights its stochastic nature and sensitivity to parameter settings.

7.2 Optimality vs Approximation Trade-offs

A key trade-off observed in this study is between optimality and approximation. BFS and A* aim to find optimal or near-optimal solutions, achieving higher ranking scores. However, these methods often produce overly filtered results, leading to very small job subsets. While optimal in score, such results are less useful for downstream analysis and decision-making.

In contrast, GA provides an approximate solution that balances multiple objectives. Although its ranking score is slightly lower, it produces more diverse and usable result sets. This makes GA more suitable for tasks requiring insight generation, pattern analysis, and multi-objective evaluation.

Overall, the results suggest that in real-world applications, approximate but scalable solutions are often more valuable than strictly optimal ones, particularly when dealing with large datasets.

Part 8. Conclusion

8.1 Summary of Key Findings

This study explored the application of BFS, A*, and GA for job retrieval and optimisation across two datasets of different scales.

The results show that:

- BFS and A* achieve the highest-ranking quality (sample dataset 1), but at the cost of high computational complexity
- A* does not significantly outperform BFS due to limited heuristic effectiveness
- GA provides a scalable and efficient solution, maintaining stable performance even for large datasets
- For large datasets, all algorithms achieve similar solution quality, making efficiency a more critical factor

Additionally, the analysis of Dataset 1 and Dataset 2 reveals a clear evolution of the job market, from general software engineering roles to specialised AI-focused positions, with increasing demand for AI frameworks, cloud technologies, and advanced technical skills.

8.2 Reflection on Applied Approaches

Overall, the applied approaches demonstrate that algorithm selection must be aligned with the problem context and scale.

For smaller datasets, BFS and A* are effective in finding optimal solutions and provide a strong baseline for evaluation. However, their lack of scalability limits their applicability in real-world scenarios.

GA, on the other hand, proves to be the most practical approach for this problem. Its ability to balance multiple objectives, handle large datasets, and produce usable results makes it well-suited for real-world job retrieval and analysis tasks. Despite its limitations, such as premature convergence and lack of guaranteed optimality, GA provides a strong trade-off between performance and solution quality.

In conclusion, this study highlights the importance of balancing optimality, scalability, and usability when designing AI search and optimisation systems. Future improvements could focus on enhancing heuristic design for A* and improving diversity and convergence behaviour in GA.

Reference

- Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly.
- Liu, P., Yuan, W., Fu, J., et al. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods. *ACM Computing Surveys*.